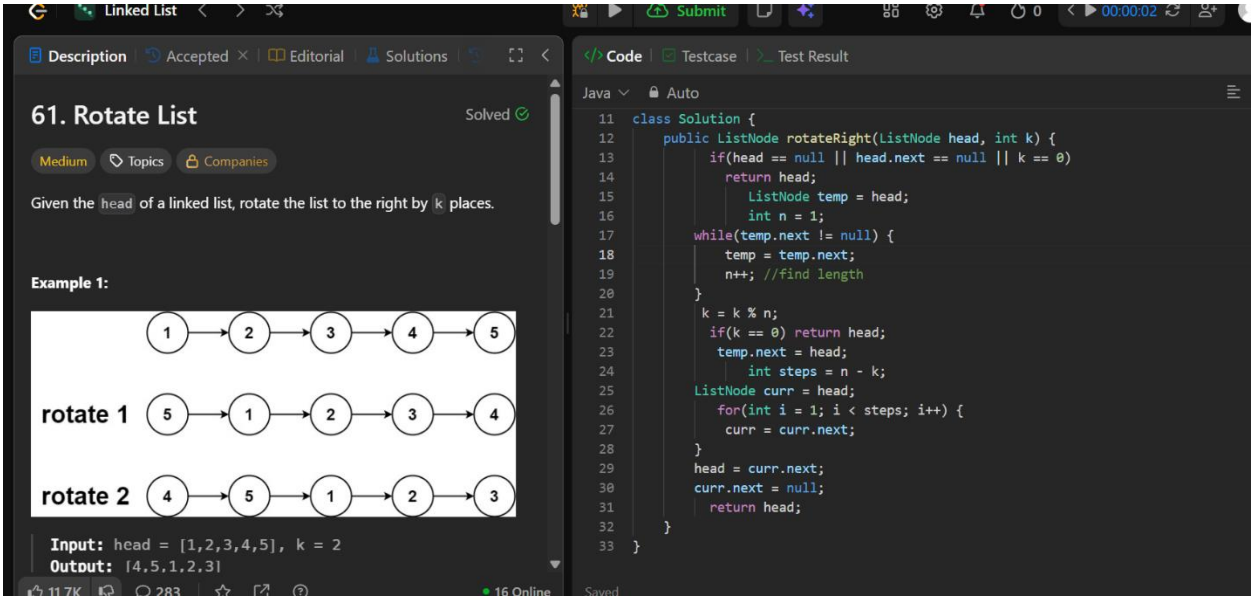
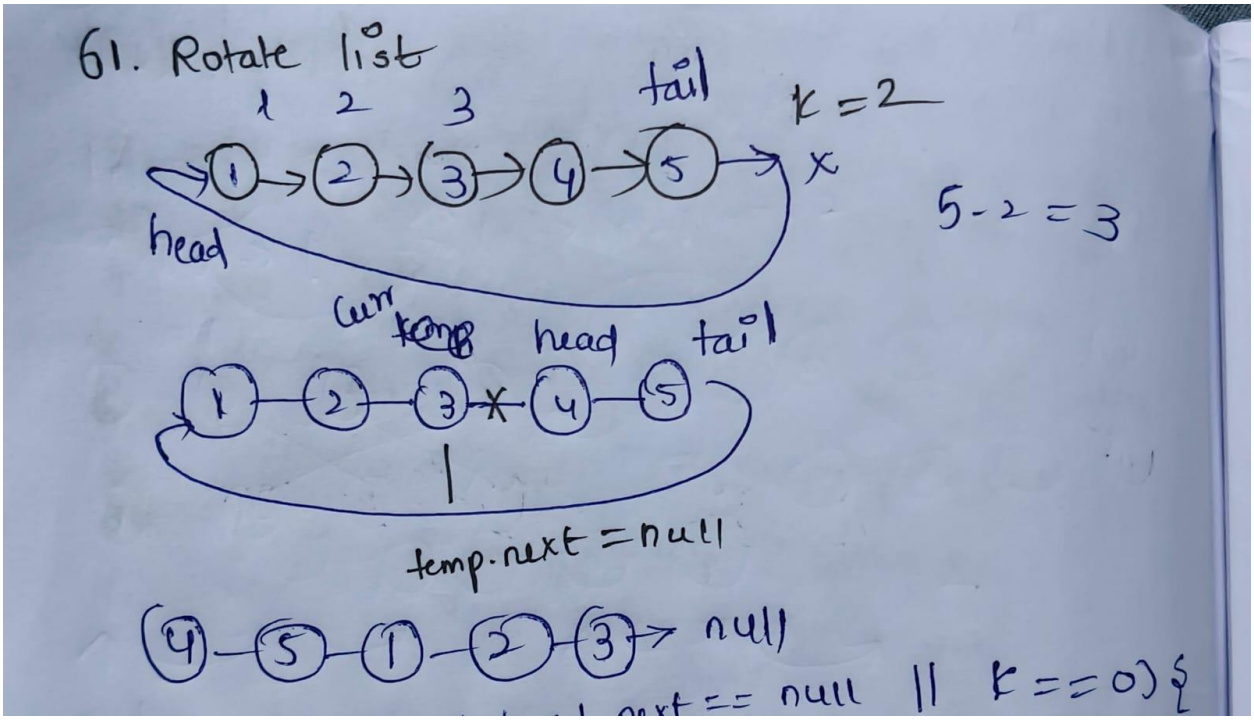
T. DHANA LAXMI
2505A41L46

1.rotate linked list

https://leetcode.com/problems/rotate-list/





2.odd even list LC-328

https://leetcode.com/problems/odd-even-linked-list/

Description | Accepted | Editorial | Solutions

328. Odd Even Linked List

Solved

Medium | Topics | Companies

Given the `head` of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*.

The **first** node is considered **odd**, and the **second** node is **even**, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

You must solve the problem in `O(1)` extra space complexity and `O(n)` time complexity.

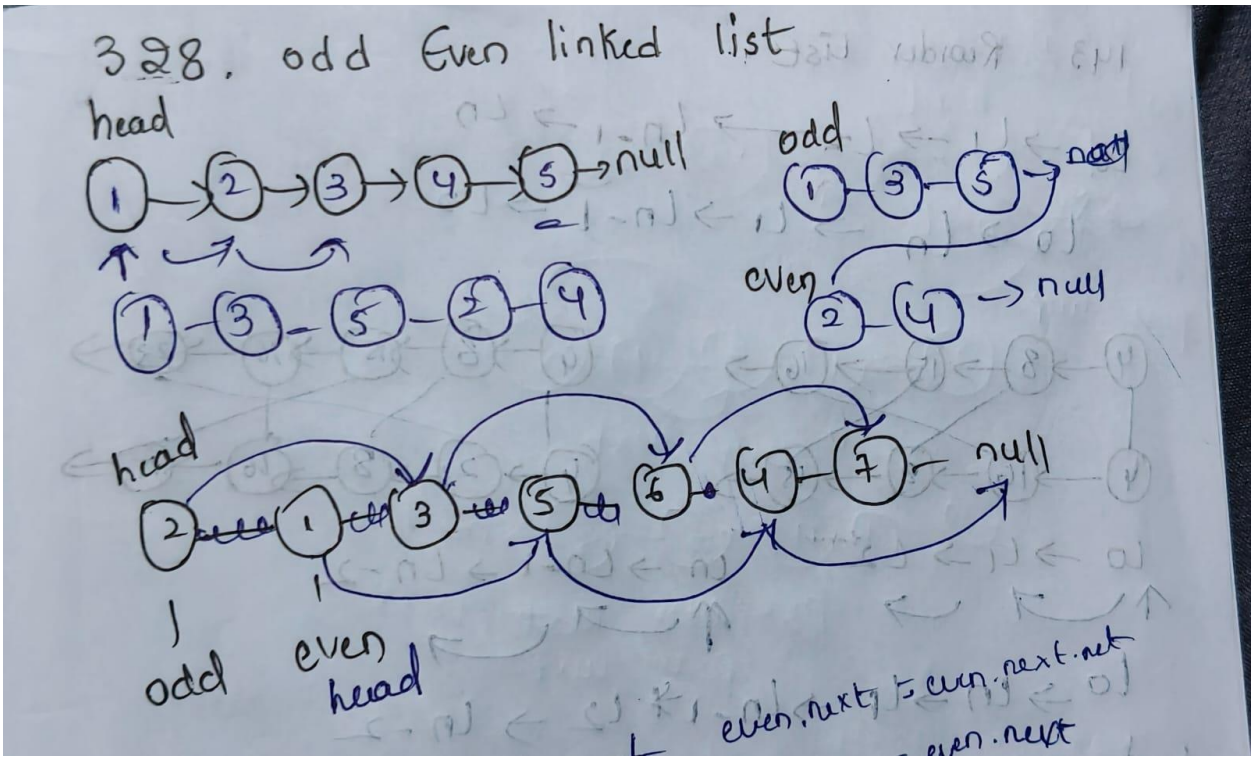
11.8K | 268 | 13 Online

Code | Testcase | Test Result

Java | Auto

```
 *     ListNode(int val, ListNode next) { this.val = val; this.next = ne
 * }
 */
class Solution {
    public ListNode oddEvenList(ListNode head) {
        if(head == null) return head;
        ListNode odd = head;
        ListNode even = head.next;
        ListNode evenhead = even;
        while(even != null && even.next != null){
            odd.next = odd.next.next;
            odd = odd.next;
            even.next = even.next.next;
            even = even.next;
        }
        odd.next = evenhead;
        return head;
    }
}
```

Saved



3.reorder list LC-143

https://leetcode.com/problems/reorder-list/

Description | Accepted | Editorial | Solutions | Submissi

143. Reorder List

Solved

Medium | Topics | Companies

You are given the head of a singly linked-list. The list can be represented as:

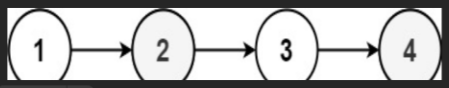
$L_0 \rightarrow L_1 \rightarrow \ldots \rightarrow L_{n-1} \rightarrow L_n$

Reorder the list to be on the following form:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \ldots$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Example 1:



12.9K | 290 | 15 Online

Code | Testcase | Test Result

Java | Auto

```java
class Solution {
    public void reorderList(ListNode head) {
        if(head == null || head.next == null) return;

        // Step 1: Find middle
        ListNode p1 = head;
        ListNode p2 = head;
        while(p2.next != null && p2.next.next != null) {
            p1 = p1.next;
            p2 = p2.next.next;
        }
        // Step 2: Separate two halves
        ListNode second = p1.next;
        p1.next = null;

        // Step 3: Reverse second half
        ListNode prev = null;
        ListNode curr = second;
            while(curr != null) {
             ListNode next = curr.next;
             curr.next = prev;
             prev = curr;
             curr = next;
```

Saved

Description | Accepted | Editorial | Solutions | Submissi

143. Reorder List

Solved

Medium | Topics | Companies

You are given the head of a singly linked-list. The list can be represented as:

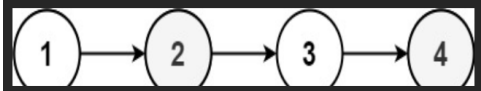
$L_0 \rightarrow L_1 \rightarrow \ldots \rightarrow L_{n-1} \rightarrow L_n$

Reorder the list to be on the following form:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \ldots$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Example 1:

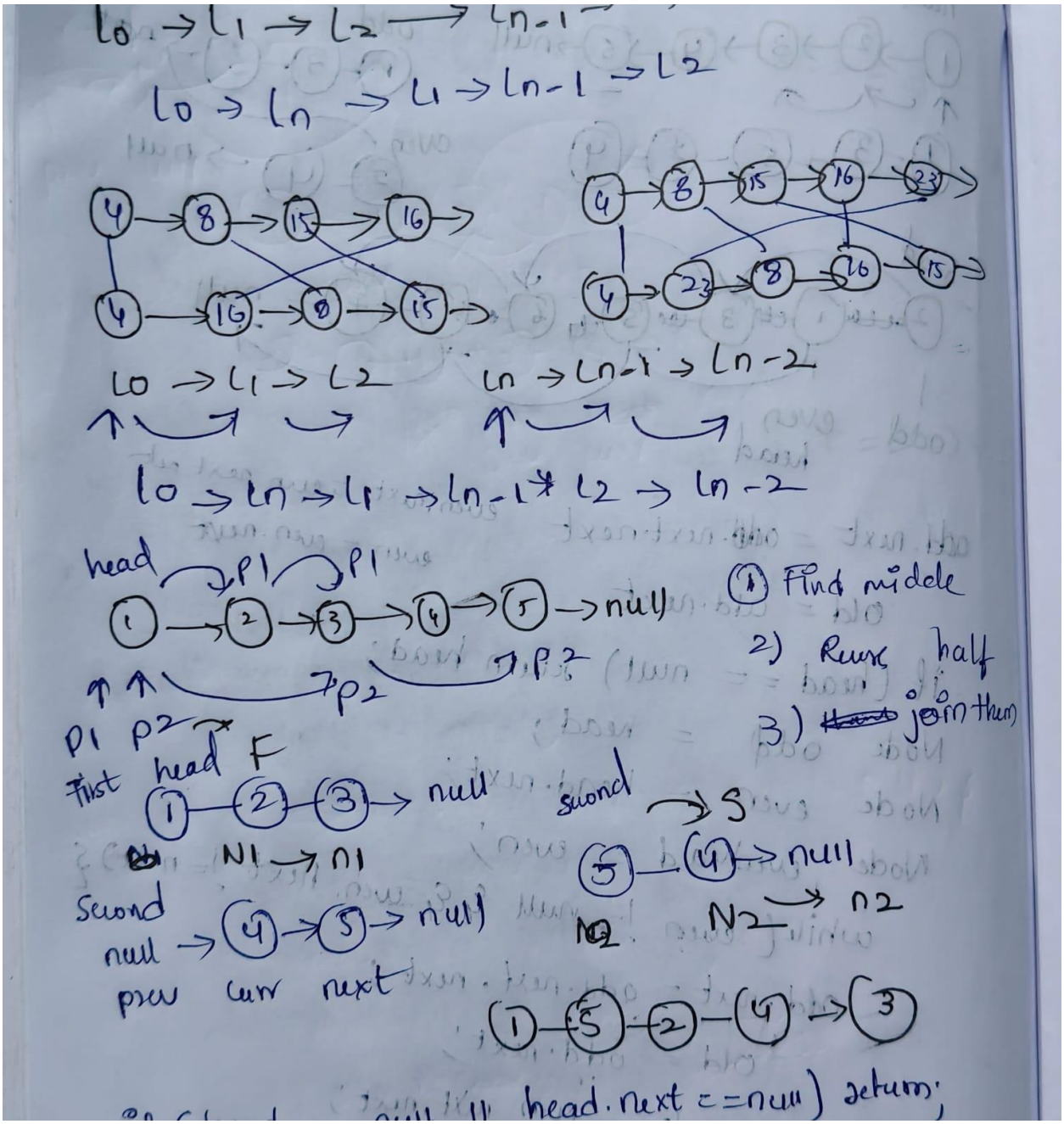


Code | Testcase | Test Result

Java | Auto

```java
        p1.next = null;

        // Step 3: Reverse second half
        ListNode prev = null;
        ListNode curr = second;
            while(curr != null) {
             ListNode next = curr.next;
             curr.next = prev;
             prev = curr;
             curr = next;
            }
            // Step 4: Merge both halves
            ListNode first = head;
            second = prev;
               while(second != null) {
                ListNode next1 = first.next;
                ListNode next2 = second.next;
                first.next = second;
                second.next = next1;
                 first = next1;
                second = next2;
            }
    }
```



4.pairwise swap in linked list (gfg)

https://www.geeksforgeeks.org/problems/pairwise-swap-elements-of-a-linked-list-by-swapping-data/

Problem | Editorial | Submissions | Comments

Pairwise Swap in Linked List Solved

Difficulty: **Easy** Accuracy: **52.06%** Submissions: **161K+** Points: **2**

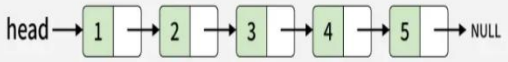
Average Time: **20m**

Given the **head** of a singly linked list, swap every two adjacent nodes and return the new head.

Note: Try to swap the nodes, not only the data.

Examples:

Input: head = 1 -> 2 -> 3 -> 4 -> 5



Output: 2 -> 1 -> 4 -> 3 -> 5

Explanation: The list has 5 nodes, so we form pairs from

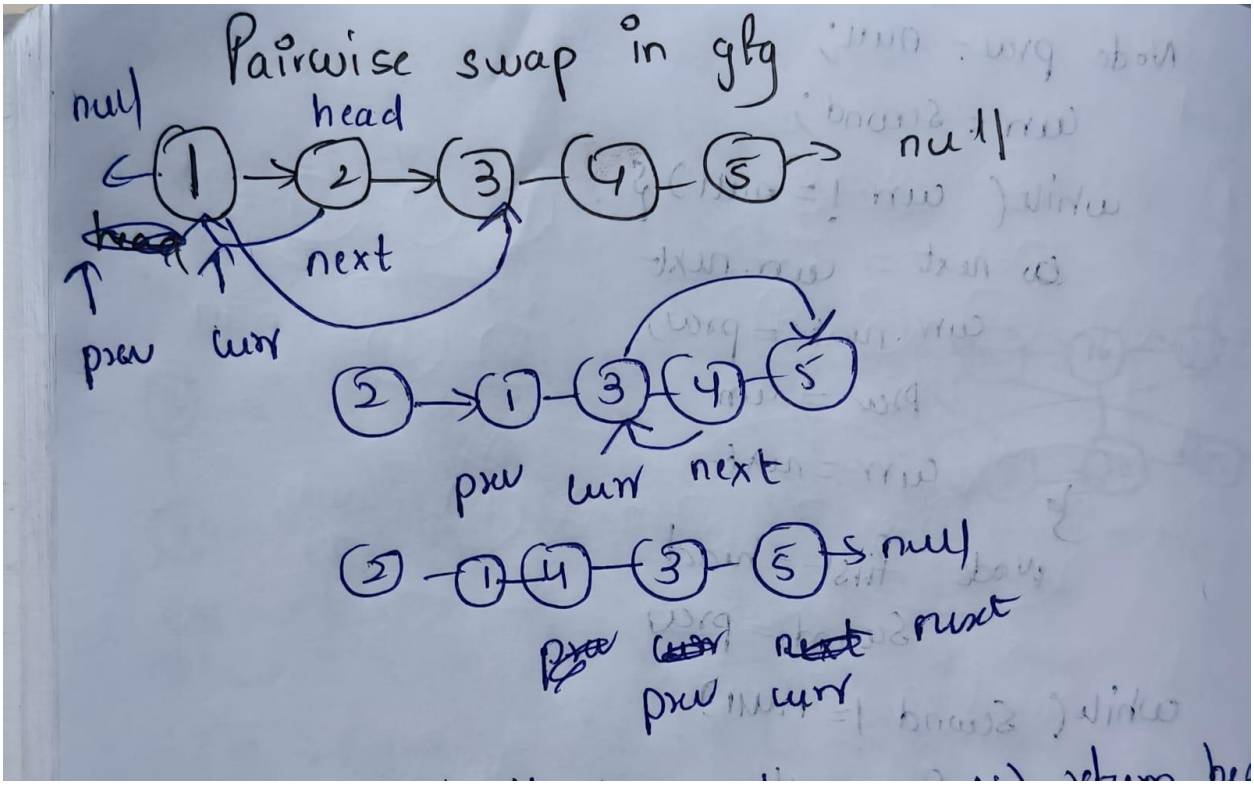
Java (21)

```
};*/

class Solution {
    public Node pairwiseSwap(Node head) {
    if(head == null || head.next == null)
            return head;

        Node prev = null;
        Node curr = head;
        head = head.next;
        while(curr != null && curr.next != null) {
            Node next = curr.next;
            curr.next = next.next;
            next.next = curr;
            if(prev != null) {
                prev.next = next;
            }
            prev = curr;
            curr = curr.next;
        }
        return head;
    }
}
```

Custom Input | Compile & Run | Su



5.flatten linked list

https://www.geeksforgeeks.org/problems/flattening-a-linked-list/1

Problem | Editorial | Submissions | Comments

Flatten Linked List

Difficulty: **Medium** Accuracy: **51.53%** Submissions: **257K+** Points: **4**
Average Time: **40m**

In a linked list, every node has two pointers: **next** and **bottom**. The heads of **n** linked lists are connected using the next pointer, while the bottom pointer points to the next node in the current linked list.

Each linked list is sorted in non-decreasing order of data, and the head nodes are also sorted in non-decreasing order.

Given the **head** of the first linked list, flatten the linked lists into a single sorted linked list such that every node is connected using only the bottom pointers.

Examples:

Input:



Java (21)

```
}*/

class Solution {
    Node merge(Node a, Node b) {
            Node dummy = new Node(-1);
        Node temp = dummy;
          while(a != null && b != null) {
           if(a.data <= b.data) {
                temp.bottom = a;
                a = a.bottom;
            }else {
                temp.bottom = b;
                b = b.bottom;
            } temp = temp.bottom;
            temp.next = null;
        }
          if(a != null) temp.bottom = a;
        else  temp.bottom = b;
         return dummy.bottom;
    }
   public Node flatten(Node head) {
       if(head == null || head.next == null)
           return head;
           head.next = flatten(head.next);
       head = merge(head, head.next);
       return head;
   }
}
```

